# TypeScript — типизация: краткий гайд

---

## Базовые типы

```
let name: string = "Alice";
let age: number = 30;
let active: boolean = true;
let nothing: null = null;
let undef: undefined = undefined;

// any — отключает проверку типов, избегай
let x: any = 42;
x = "oops"; // TS не ругается — плохо

// unknown — безопасная альтернатива any
let val: unknown = getData();
if (typeof val === "string") val.toUpperCase(); // OK — сначала проверяем
```

---

## Массивы и кортежи

```
// Массив
const nums: number[] = [1, 2, 3];
const strs: Array<string> = ["a", "b"];

// Кортеж — фиксированная длина и порядок типов
const pair: [string, number] = ["age", 30];
const rgb: [number, number, number] = [255, 128, 0];
```

---

## Объекты — интерфейс и тип

```
// Interface — для объектов и классов, расширяемый
interface User {
  id: number;
  name: string;
  email?: string; // необязательное поле
}

// Type — гибче, можно делать union/intersection
type Point = { x: number; y: number };
type ID = string | number; // union type
```

Когда interface, когда type

| Возможность | interface | type |
|---|---|---|
| Объекты/классы | ✅ предпочтительно | ✅ |
| Union/Intersection | ❌ | ✅ |
| Расширение (extends) | ✅ удобнее | ✅ (&) |
| Можно переоткрыть | ✅ | ❌ |

---

## Union и Intersection

```
// Union — одно ИЗ
type StringOrNumber = string | number;
function format(val: StringOrNumber) { ... }

// Discriminated union — с признаком
type Shape =
  | { kind: "circle"; radius: number }
  | { kind: "rect"; width: number; height: number };

function area(s: Shape): number {
  if (s.kind === "circle") return Math.PI * s.radius ** 2;
  return s.width * s.height;
}

// Intersection — всё СРАЗУ
type Admin = User & { role: string };
```

---

## Функции

```
// Типизация параметров и возврата
function add(a: number, b: number): number {
  return a + b;
}

// Arrow function
const greet = (name: string): string => `Hello, ${name}`;

// Необязательный и дефолтный параметр
function log(msg: string, prefix?: string): void {
  console.log(prefix ?? "", msg);
}

// Rest параметры
function sum(...nums: number[]): number {
```

```
  return nums.reduce((a, b) => a + b, 0);
}
```

---

## Generics — обобщённые типы

Generics — параметры типов, как переменные но для типов.

```
// Функция с generics
function identity<T>(val: T): T {
  return val;
}
identity<number>(42);   // T = number
identity("hello");      // T = string (TS выводит сам)

// Generic функция — обёртка массива
function first<T>(arr: T[]): T | undefined {
  return arr[0];
}

// Generic интерфейс
interface Box<T> {
  value: T;
  label: string;
}
const box: Box<number> = { value: 42, label: "num" };

// Ограничение через extends
function getLength<T extends { length: number }>(val: T): number {
  return val.length;
}
getLength("hello"); // 5
getLength([1, 2, 3]); // 3
// getLength(42); // ❌ Error — у числа нет length
```

---

## Утилитарные типы

```
interface User { id: number; name: string; email: string }

// Partial — все поля необязательные
type UserDraft = Partial<User>;
// { id?: number; name?: string; email?: string }

// Required — все поля обязательные
type FullUser = Required<UserDraft>;

// Readonly — запрет изменений
type FrozenUser = Readonly<User>;
```

```
const u: FrozenUser = { id: 1, name: "Al", email: "a@b.c" };
// u.id = 2; // ❌ Error

// Pick — выбрать поля
type UserPreview = Pick<User, "id" | "name">;

// Omit — исключить поля
type UserWithoutEmail = Omit<User, "email">;

// Record — объект с фиксированным типом ключей и значений
type Roles = Record<string, "admin" | "user" | "guest">;
const roles: Roles = { alice: "admin", bob: "user" };

// ReturnType — тип возврата функции
function getUser() { return { id: 1, name: "Alice" }; }
type UserReturn = ReturnType<typeof getUser>; // { id: number; name:
string }
```

---

## Type Guards — сужение типов

```
// typeof
function process(val: string | number) {
  if (typeof val === "string") {
    return val.toUpperCase(); // здесь val: string
  }
  return val.toFixed(2);     // здесь val: number
}

// instanceof
function handleDate(val: Date | string) {
  if (val instanceof Date) return val.getFullYear();
  return new Date(val).getFullYear();
}

// Custom type guard — функция-предикат
function isString(val: unknown): val is string {
  return typeof val === "string";
}

// Narrowing discriminated union
function area(s: Shape): number {
  switch (s.kind) {
    case "circle": return Math.PI * s.radius ** 2;
    case "rect":   return s.width * s.height;
  }
}
```

---

## as и satisfies

```
// as — утверждение типа (используй осторожно!)
const input = document.getElementById("name") as HTMLInputElement;
input.value; // OK — мы сказали TS что это HTMLInputElement

// satisfies — проверяет тип без приведения
const config = {
  host: "localhost",
  port: 3000,
} satisfies Record<string, string | number>;

config.host.toUpperCase(); // OK — TS знает что host это string
```

---

## Readonly и const assertions

```
// as const — замораживает значение и тип
const direction = ["up", "down", "left", "right"] as const;
type Direction = (typeof direction)[number]; // "up" | "down" | "left" |
"right"

const config = { env: "prod", port: 3000 } as const;
// config.env: "prod"  (не string — литеральный тип)
```

---

## Иерархия типов — кто кому assignable

Стрелка = "является подтипом" / "assignable to" (снизу вверх).

```
graph BT
    never(["never"])

    any_top(["any / unknown"])
    obj["{} / Object"]

    object["object"]
    Symbol["Symbol"]
    String["String"]
    Number["Number"]
    Boolean["Boolean"]

    symbol["symbol"]
    string["string"]
    number["number\nnumber enum"]
    boolean["boolean"]
    void_t["void"]
    null_t["null"]

    unique_sym["unique symbol"]
```

```
    str_enum["string enum"]
    undef["undefined"]

    ro_arr["readonly Array"]
    ro_tup["readonly tuple"]
    arr["Array"]
    func["Function"]
    tup["tuple"]

    %% never → всё
    never --> null_t
    never --> undef
    never --> void_t
    never --> boolean
    never --> string
    never --> number
    never --> symbol
    never --> unique_sym
    never --> str_enum
    never --> arr
    never --> func
    never --> tup
    never --> ro_tup
    never --> ro_arr
    never --> object
    never --> any_top

    %% {} / Object
    object --> obj
    Symbol --> obj
    String --> obj
    Number --> obj
    Boolean --> obj

    %% wrappers → {}
    obj --> any_top

    %% primitives → wrappers
    symbol --> Symbol
    string --> String
    number --> Number
    boolean --> Boolean

    %% literals / enums
    unique_sym --> symbol
    str_enum --> string

    %% null/undefined/void
    null_t --> any_top
    void_t --> any_top
    undef --> void_t
    undef --> null_t

    %% arrays
```

```
    ro_arr --> object
    arr --> ro_arr
    ro_tup --> ro_arr
    tup --> ro_tup
    tup --> arr
    func --> object
```

Ключевые правила

| Тип | Assignable to |
|---|---|
| `never` | всё (bottom type) |
| `any` | всё и из всего (обходит проверки) |
| `unknown` | только `unknown` и `any` без сужения |
| `undefined` | `void`, `null` (и любой `T \| undefined`) |
| `null` | `unknown`, `any` (в strict режиме — только они) |
| `string enum` | `string` |
| `number enum` | `number` |
| `unique symbol` | `symbol` |
| `tuple` | `Array`, `readonly Array`, `readonly tuple` |
| примитивы (`string`, `number`...) | wrapper-классы (`String`, `Number`...) → `{}` → `object` |

> `{}` принимает всё кроме `null` и `undefined` (в `strictNullChecks`).

---

## Быстрая шпаргалка

| Конструкция | Пример |
|---|---|
| Базовый тип | `let x: number` |
| Опциональное поле | `name?: string` |
| Union | `string \| number` |
| Intersection | `A & B` |
| Generic | `function f<T>(x: T): T` |
| Сужение | `if (typeof x === "string")` |
| Partial | `Partial<User>` |
| Pick | `Pick<User, "id" \| "name">` |
| Omit | `Omit<User, "email">` |
| Record | `Record<string, number>` |

| Конструкция | Пример |
|---|---|
| as const | `["a", "b"] as const` |